

ASP.NET

Unit-1

❖ ASP.NET Introduction

- ASP.NET is a web application framework developed and marketed by Microsoft to allow programmers to build dynamic web sites, web applications and web services.
- .NET is a free, open-source developer platform created by Microsoft for building various types of applications, including web, mobile, desktop, cloud, and games.
- ASP.NET, the next version of ASP, is a programming framework used to create enterprise-class Web Applications.
- It was first released in January 2002 with version 1.0 of the .NET Framework
- ASP.NET is built on the Common Language Runtime (CLR), allowing programmers to write ASP.NET code using any supported .NET language.
- ASP.NET allows you to use a full featured programming language such as C# (pronounced C-Sharp) or VB.NET to build web applications easily.
- Both ASP and ASP.NET allow a Web site builder to dynamically build Web pages by inserting queries to a relational database in the Web page.
- ASP.NET files can be recognized by their .aspx extension.

Comparison .Net technology and over other current technology.

- .NET technology is a strong contender in the current tech landscape, offering a robust framework for building various applications with strong performance and integration capabilities, particularly within the Microsoft ecosystem.
- While other technologies like Java offer cross-platform compatibility and a broader industry backing, .NET's performance, especially in cloud environments, and its rich tooling ecosystem make it a compelling choice for many developers.
- **.NET technology** against other current popular technologies in software development, particularly in **web, desktop, mobile, and enterprise** applications. For development ecosystem, web development, Mobile development, Desktop development, Cloud & enterprise use, and community & support.

Advantages of .NET Technology

1. Cross-Platform Development (.NET Core/.NET 5+)

- Build and run apps on **Windows, Linux, and macOS**.
- Same codebase for different platforms using **.NET** (for mobile & desktop) or **ASP.NET Core** (for web).

2. High Performance & Scalability

- Compiled to **native code** and optimized for speed.
- Handles high-load web apps, APIs, and enterprise services.
- Supports **asynchronous programming** and **microservices**.

3. Modern Programming Language – C#

- C# is powerful, type-safe, and modern with features like:
 - LINQ (language-integrated queries)
 - Async/await
 - Pattern matching
 - Records and value types

4. Robust Security

- In-built support for **authentication, authorization, data protection, and role-based access control**.
- Integrates well with **OpenID Connect, Active Directory, and Azure AD**.

5. Comprehensive Development Tools

- Industry-standard tools like **Visual Studio, Visual Studio Code**.
- Strong debugger, profiler, and integration with **CI/CD pipelines**.

6. Strong Support for Cloud and Micro services

- Deep integration with **Azure**, and excellent support for **Docker** and **REST APIs**.
- Ideal for building **cloud-native, server less, and containerized** applications.

7. Large Ecosystem and Package Manager

- Huge collection of libraries and packages through **Get**. Rich APIs for web, desktop, mobile, data access, machine learning, etc.

8. Integrated – Entity Framework Core

- Simplifies database access with **code-first** and **database-first** approaches.
- Supports **LINQ, lazy loading, and automatic migrations**.

9. Backed by Microsoft and Open Source

- Strong corporate backing with long-term support (LTS) versions.
- Open source under the .NET Foundation with active community contributions.

10. Wide Industry Adoption

- Common in **enterprise, banking, healthcare, and government** systems.
- Large pool of skilled developers and job opportunities.

Comparison & Overview of all .net frameworks

- .NET is the modern, cross-platform, open-source successor to the .NET Framework, which is Windows-only. .NET Framework is included with Windows and automatically updated, while .NET is released independently and requires manual installation.

.NET (formerly .NET Core)

- **Cross-platform:** Runs on Windows, macOS, and Linux.
- **Open-source:** Actively developed and maintained by the community.
- **Modular and lightweight:** Allows developers to include only necessary components.
- **Improved performance and scalability:** Offers better performance and scalability compared to .NET Framework.
- **ASP.NET Core:** Modern framework for building web applications.
- **Entity Framework Core:** Modern framework for data access.
- **Side-by-side installation:** Allows multiple .NET versions on the same machine.
- **Modern development:** The recommended platform for new development.

.NET Framework

- **Windows-only:** Designed to run exclusively on Windows.
- **Proprietary:** While source code is available, it doesn't accept community contributions.
- **Monolithic:** All components are included in a single package.
- **Mature and stable:** Widely used and well-established, but no longer the focus of new development.
- **ASP.NET:** Legacy framework for building web applications.
- **Entity Framework:** Legacy framework for data access.

- **Not side-by-side:** Only one version of .NET Framework can exist on a machine at a time.
- **For existing applications:** Best for maintaining existing applications built with the .NET Framework.

.Net Framework Versions:

- Microsoft started development on the .NET Framework in the late 1990s originally under the name of Next Generation Windows Services (NGWS).

Version	Version Number	Release Date	Visual Studio	Default in Windows
1.0	1.0.3705.0	2002-02-13	Visual Studio .NET	
1.1	1.1.4322.573	2003-04-24	Visual Studio .NET 2003	Windows Server 2003
2.0	2.0.50727.42	2005-11-07	Visual Studio 2005	
3.0	3.0.4506.30	2006-11-06		Windows Vista, Windows Server 2008
3.5	3.5.21022.8	2007-11-19	Visual Studio 2008	Windows 7, Windows Server 2008 R2
4.0	4.0.30319.1	2010-04-12	Visual Studio 2010	
4.6		02-08-2016		
4.7		05-04-2017		
4.7.2		30-04-2018		
4.8		18-04-2019		
4.8.1		09-08-2022		

Introduction about Applications to be developed by .Net:

- **.NET Framework:** The original implementation, primarily for Windows applications.
- **.NET (formerly .NET Core):** A cross-platform, open-source version for building applications on Windows, Linux, and macOS.
- **Cross-Platform Support:** .NET allows developers to build applications that can run on different operating systems, including Windows, macOS, Linux, iOS, and Android.
- **Application Types:** .NET supports the development of various application types:
 - **Web Applications:** Using ASP.NET, developers can build dynamic websites, web services, and web APIs.
 - **Desktop Applications:** Windows Forms and WPF (Windows Presentation Foundation) are used to create traditional desktop applications.
 - **Mobile Applications:** An implementation of .NET, enables the development of cross-platform mobile apps for iOS and Android.

- **Game Development:** .NET supports game development with frameworks like Unity.
- **Cloud Applications:** .NET is well-suited for building cloud-based applications and services.
- **Console Applications:** Lightweight applications that run in the command line.
- **Windows Services:** Applications that run in the background on Windows.
- **Programming Languages:** C#, VB.NET, and F# are the primary languages used with .NET.
- **.NET Ecosystem:** A rich set of libraries, tools (like Visual Studio and VS Code), and community resources supports .NET development.
- **Open Source:** .NET is open-source, allowing for community contributions and transparency.

❖ The .Net Framework Architecture:-

- The **.NET Framework architecture** is a **software development platform** developed by Microsoft that provides a controlled environment for developing and running applications. It supports multiple languages (like C#, VB.NET, J#) and enables the building of Windows, web, and enterprise applications.

❖ Components of .Net Framework

- Common Language Runtime. (CLR)
- .NET Framework Class Library. (FCL)
- Common Language Specification. (CLS)
- Common Type System. (CTS)
- Metadata and Assemblies.
- Manage Code and Unmanaged Code.
- Windows Forms.
- ASP.NET and ASP.NET AJAX.
- ADO.NET
- Windows Presentation Foundation. (WPF)
- Windows Communication Foundation. (WCF)
- Windows Workflow Foundation. (WF).
- Windows CardSpace.

❖ Role of CLR in .Net Framework:-

1. Code Execution (JIT Compilation)

- Converts **MSIL (Microsoft Intermediate Language)** code into **native machine code** using the **Just-In-Time (JIT) Compiler**.
- Executes the compiled native code on the host machine.

2. Memory Management

- Allocates and deallocates memory automatically.
- Uses **Garbage Collection (GC)** to release memory that is no longer in use.

3. Garbage Collection (GC)

- Periodically checks for unused objects in memory and clears them.
- Improves performance and avoids manual memory handling.

4. Security Management

- Provides two levels of security:
 - **Code Access Security (CAS)** – Controls what code can do based on evidence.
 - **Role-Based Security** – Access control based on user roles.

5. Exception Handling

- Provides a structured model for error detection and recovery using try, catch, finally blocks.
- Ensures consistency in exception handling across languages.

6. Language Interoperability

- Allows code written in different .NET languages (C#, VB.NET, F#) to work together.
- Ensures compatibility through **CTS** and **CLS**.

7. Assembly Loading & Management

- Loads and manages **assemblies (EXE/DLL files)**.
- Verifies version, metadata, and security.

8. Type Safety and Verification

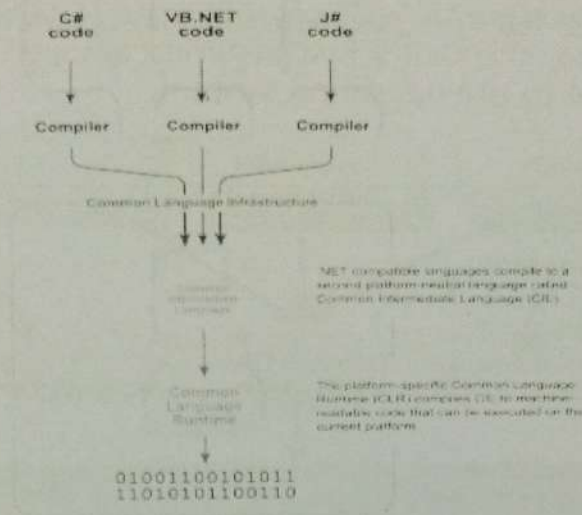
- Ensures that code only accesses memory it is authorized to.
- Prevents illegal type casts or buffer overflows.

9. Thread Management

- Manages threads and thread pool for efficient multitasking and parallelism.
- Supports synchronization and concurrency.

❖ Introduction about Language supported by .Net:-

- The Microsoft .NET Framework is a software framework that can be installed on computers that running Microsoft Windows operating systems.
- It includes a large library of coded solutions to common programming problems and a virtual machine that manages the execution of programs written specifically for the framework.

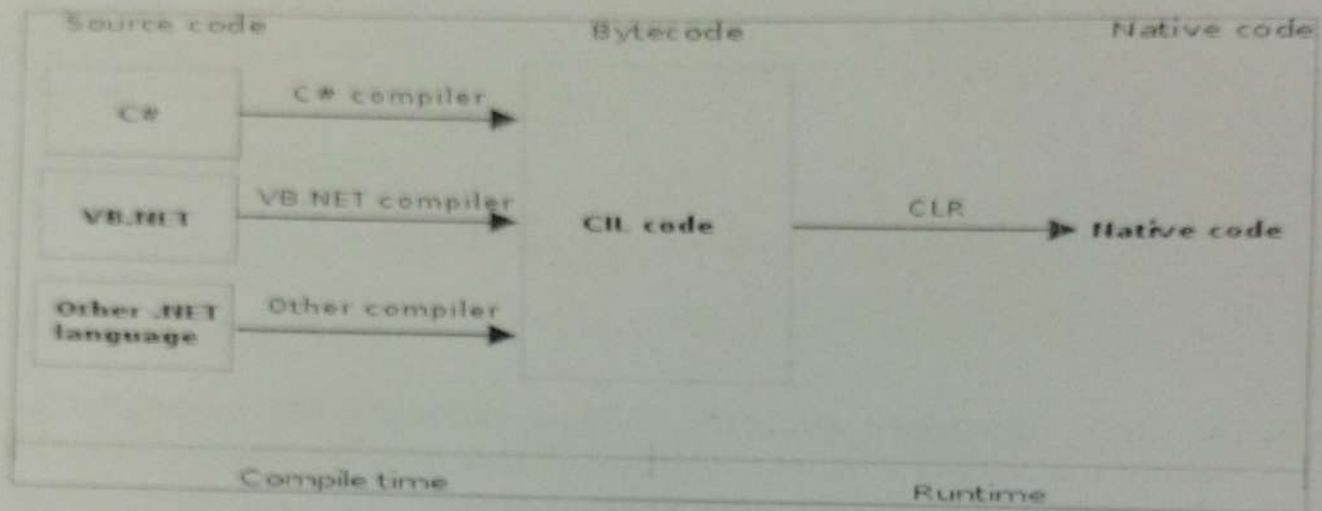


- The .NET framework supports multiple programming languages in a manner that allows language interoperability, whereby each language can utilize code written in other languages
- The framework's Base Class Library provides a large range of features including user interface, data access, database connectivity, cryptography, web application development, numeric algorithms, and network communications.
- The class library is used by programmers, who combine it with their own code to produce applications.
- Programs written for the .NET Framework execute in a software environment that **manages the program's runtime requirements.**
- This runtime environment is known as the Common Language Runtime (CLR).

❖ Common Language Runtime. (CLR Architecture)

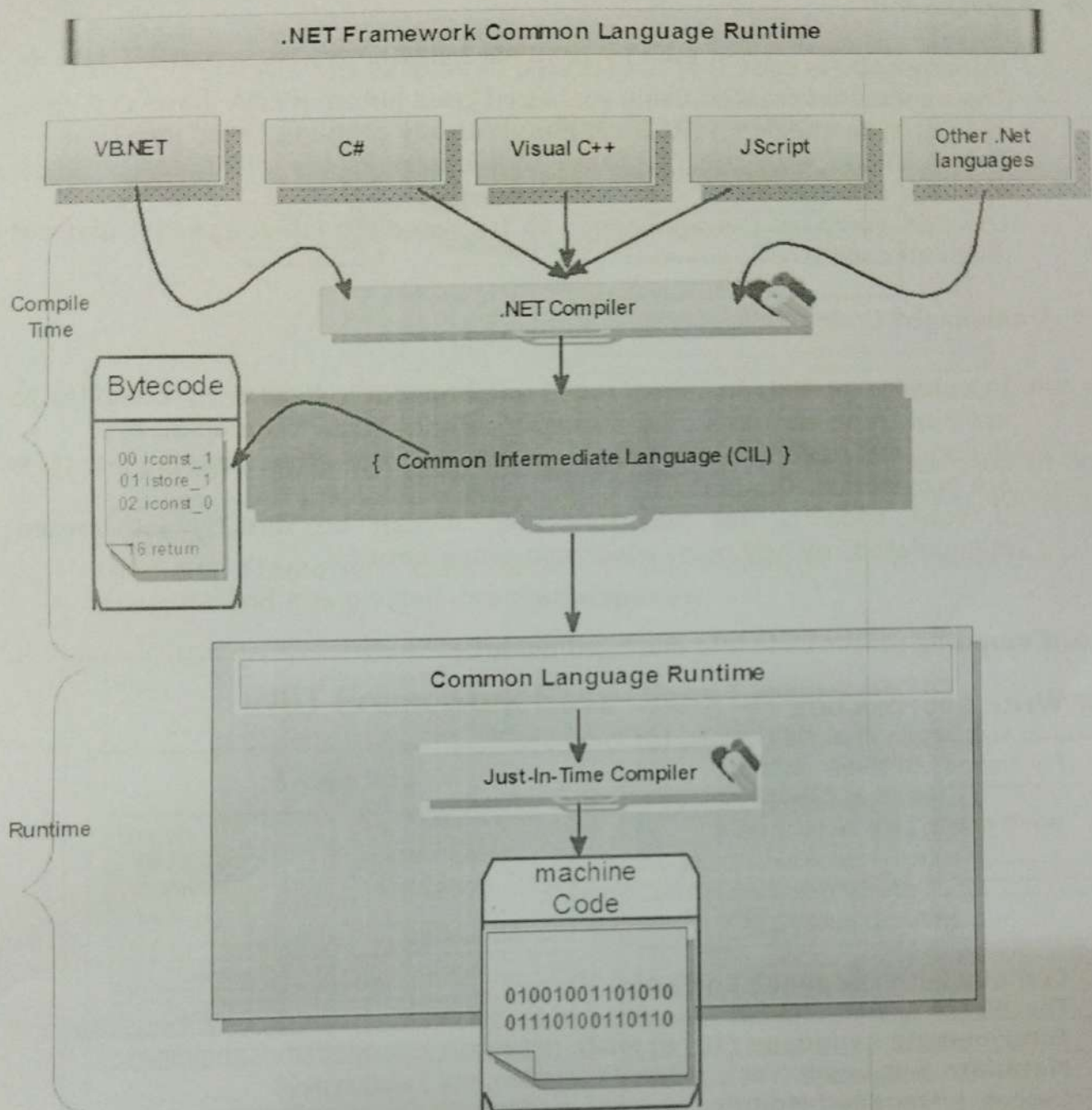
- The .NET Framework provides a run-time environment called the common language runtime.
- CLR runs the code and provides services that make the development process easier.
- The Common Language Runtime manages the execution of the code.
- It provides the functionalities, such as memory management, exception handling, debugging, security, thread execution, code execution, code safety, verification, compilation.
- The code that works on the CLR is called the managed code.
- The CLR automatically release the objects when they are no longer in use. This automatically memory management is known as Garbage Collection.
- It is Microsoft's implementation of the Common Language Infrastructure (CLI) standard.
- Developers using the CLR write code in a language such as C# or VB.NET.

- **At compile time**, a .NET compiler converts such code into CIL code.
- **At runtime**, the CLR's just-in-time compiler converts the CIL code into code native to the operating system.



- A Piece of Managed code is executed as follows:

1. Choosing language compiler.
2. Compiling the code to IL.
3. Compiling IL to native code.
4. Executing the code.



- When you compile the code into managed code, the compiler converts the source code into IL(Intermediate Languages OR CIL (common)),
- Before the execution of the code, IL must be converted into CPU-Specific code by the JUST-IN-TIME (JIT) compiler.
- When you compile your source code into IL, the required metadata is generated.
- Before running the IL, it must be converted to native code. The native code is CPU-specific code which is nothing but the output of the JIT compiler, that is, the machine code that actually runs in the runtime.

❖ Manage Code.

- Manage code is code that is executed directly by the CLR.
- The application created using managed code automatically have CLR services, such as type checking, security, automatically garbage collection.
- This IL along with the metadata that describes the attributes, classes and methods of the code.
- The CLR compiles the application to Intermediate Language (IL) and not the machine code.

❖ Unmanaged Code.

- In contrast to the managed code, unmanaged code directly compiles to the machine code and runs on the machine where it has been compiled.
- It does not have services, such as security or memory management, which are provided by the CLR.
- If your code is not security-prone, it can be directly interpreted (or Manipulated) by any user, which can prove harmful.

❖ Compiling source code into managed module:-

1. Write Source Code

- You write code in a .NET-supported language.
- For Ex.

```
// Example in Csharp#  
public class HelloWorld {  
    public static void Main() {  
        Console.WriteLine("Hello, World!");  
    }  
}
```

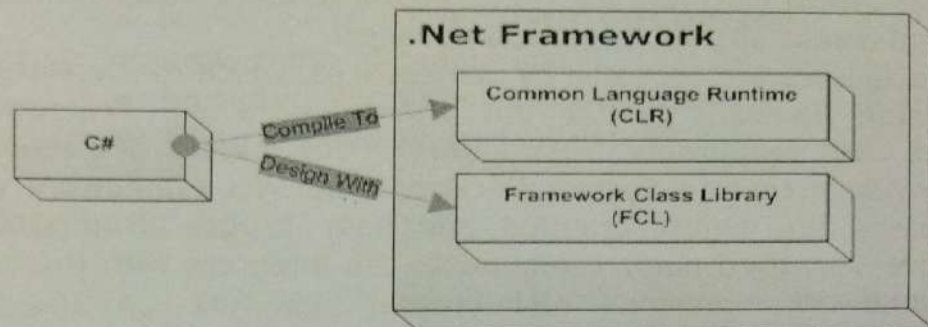
2. Compile with Language Compiler

- The .NET language compiler (like **csc.exe** for C#) converts the source code into.
- **Intermediate Language (IL)** or **MSIL** (Microsoft Intermediate Language).
- **Metadata** (info about types, methods, version, etc.)
- Output: A **Managed Module** (.dll or .exe file)
- C# Source Code → [C# Compiler] → Managed Module (.dll/.exe)
- This **managed module** contains **IL + metadata**, and is **not yet native machine code**.

3. What Is a Managed Module?

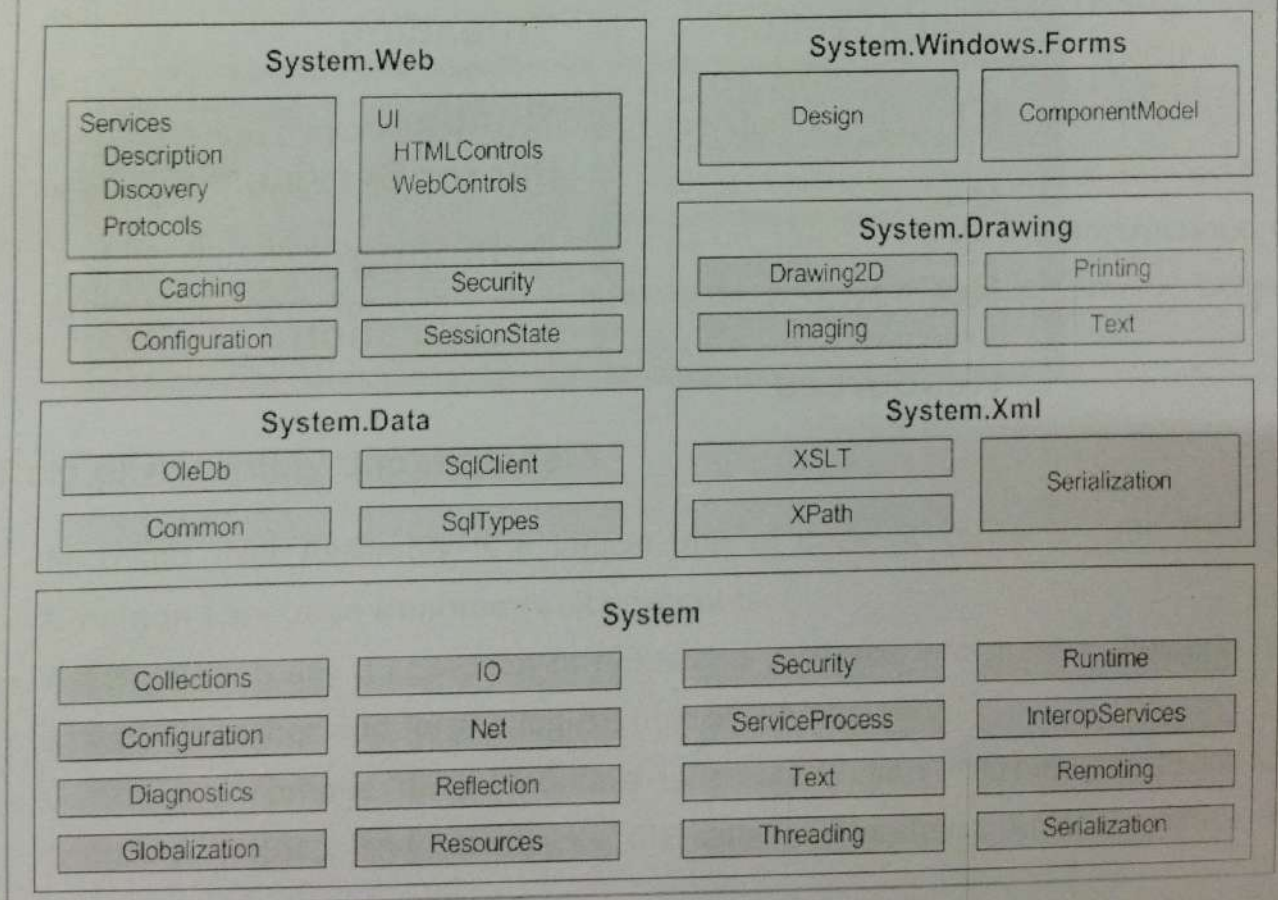
- A **managed module** is a portable executable (PE) file that includes.
- **MSIL code** (CPU-independent instructions).
- **Metadata** (type definitions, method signatures, etc.)
- **Manifest** (info about the assembly like version, culture, references) It's called "managed" because it will be run **under the control of the CLR**.

❖ .NET Framework Class Library. (FCL)



- The .NET Framework class library is a library of classes, interfaces, and value types that are included in the Microsoft .NET Framework SDK.
- This library provides access to system functionality and is designed to be the foundation on which .NET Framework applications, components, and controls are built.
- The .Net Framework class library (FCL) organized in a hierarchical tree structure and it is divided into **Namespaces**.

.NET Framework Base Class Library (BCL)



.NET Framework Namespaces

Namespaces is a logical grouping of types for the purpose of identification. Framework class library (FCL) provides the consistent base types that are used across all .NET enabled languages. The Classes are accessed by namespaces, which reside within Assemblies. The System Namespace is the root for types in the .NET Framework. The .Net Framework class library (FCL) classes are managed classes that provide access to System Services. The .Net Framework class library (FCL) classes are object oriented and easy to use in program developments. Moreover, third-party components can integrate with the classes in the .NET Framework.

Base Class Library Namespaces

System	
Collections	Security
Configuration	ServiceProcess
Diagnostics	Text
Globalization	Threading
IO	Runtime
Net	InteropServices
Reflection	Remoting
Resources	Serialization

- Data types, conversions, formatting
- Collections: ArrayList, Hashtable, etc.
- Globalization: Cultures, sorting, etc.
- I/O: Binary and text streams, files, etc.
- Networking: HTTP, TCP/IP sockets, etc.
- Reflection: Metadata and IL
- Security: Permissions, cryptography
- Text: Encodings, regular expressions

❖ Intermediate Language

- MSIL stands for Microsoft Intermediate Language. We can call it as Intermediate Language (IL) or Common Intermediate Language (CIL).
- During the compile time, the compiler converts the source code into Microsoft Intermediate Language (MSIL). Microsoft Intermediate Language (MSIL) is a CPU-independent set of instructions that can be efficiently converted to the native code. During the runtime the Common Language Runtime (CLR)'s Just In Time (JIT) compiler converts the Microsoft Intermediate Language (MSIL) code into native code to the Operating System.
- When a compiler produces Microsoft Intermediate Language (MSIL), it also produces Metadata. The Microsoft Intermediate Language (MSIL) and Metadata are contained in a portable executable (PE) file.
- Microsoft Intermediate Language (MSIL) includes instructions for loading, storing, initializing, and calling methods on objects, as well as instructions for arithmetic and logical operations, control flow, direct memory access, exception handling, and other operations.

❖ Role of Assembly and metadata.

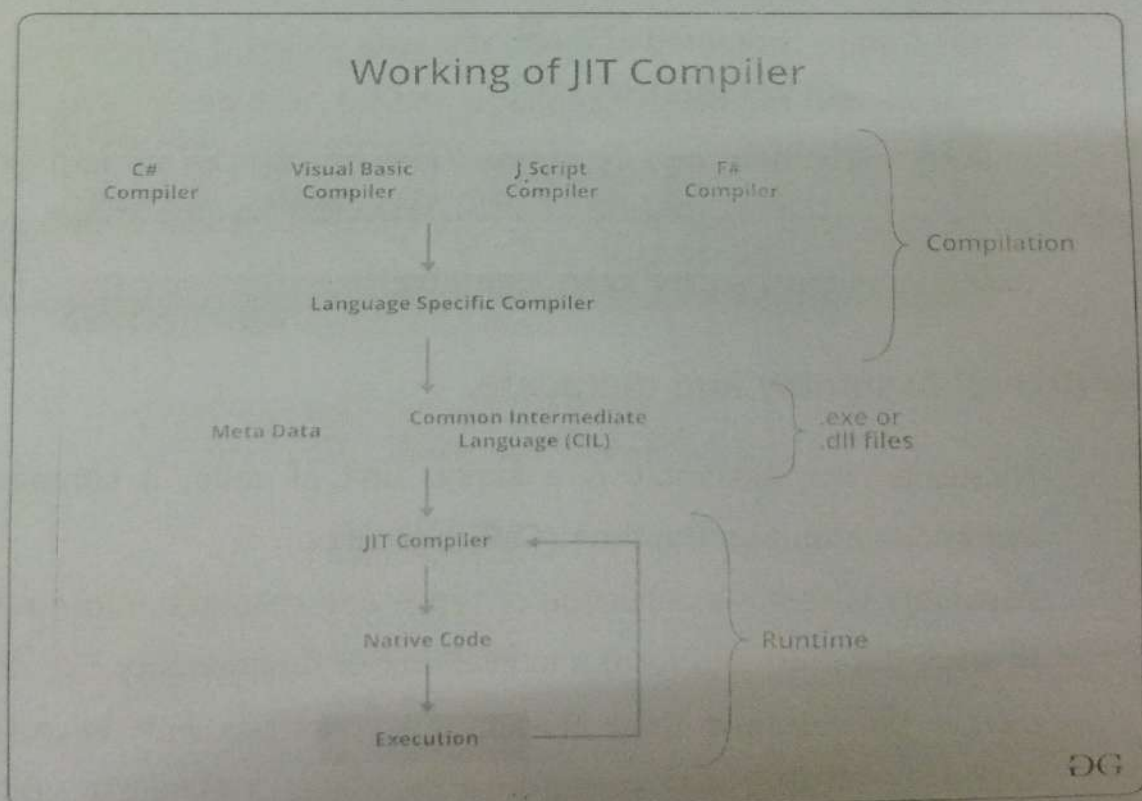
- Microsoft .Net Assembly is a logical unit of code, it contains code that the Common Language Runtime (CLR) executes.
- Assembly is really a collection of types and resource information that are built to work together and form a logical unit of functionality.
- During the compile time Metadata is created, with Microsoft Intermediate Language (MSIL), and stored in a file called a Manifest. Both Metadata and

Microsoft Intermediate Language (MSIL) together wrapped in a Portable Executable (PE) file.

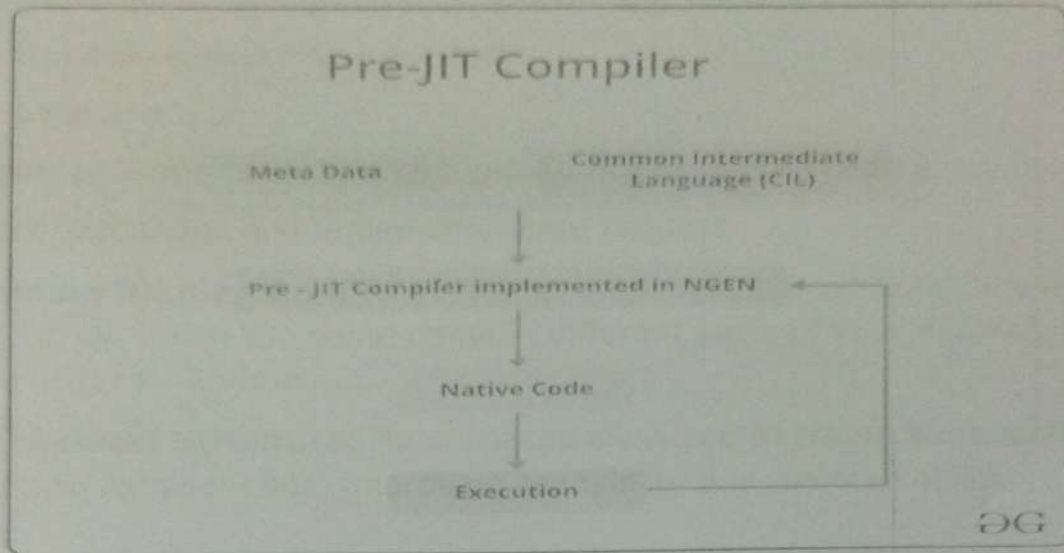
- Manifest contains information about itself. This information is called Assembly Manifest, it contains information about the members, types, references and all the other data that the runtime needs for execution.
- Every Assembly you create contains one or more program files and a Manifest. There are two types program files : Process Assemblies (EXE for Executable File) and Library Assemblies (DLL).

❖ Just In Time Compiler

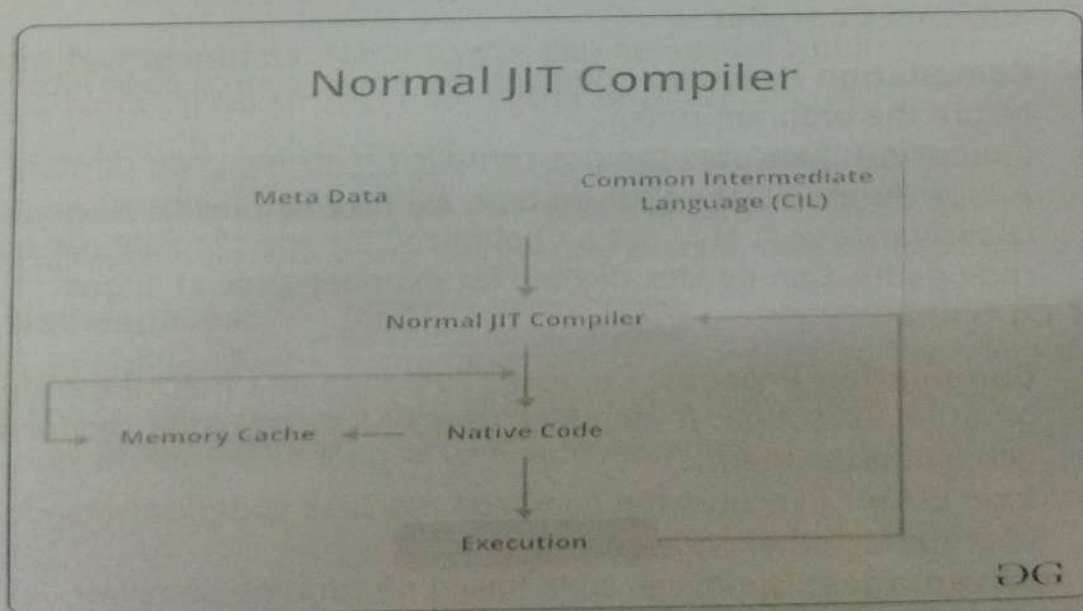
- The .Net languages, which is conforms to the Common Language Specification (CLS), uses its corresponding runtime to run the application on different Operating Systems.
- During the code execution time, the Managed Code compiled only when it is needed, that is it converts the appropriate instructions to the native code for execution just before when each function is called. This process is called Just In Time (JIT) compilation, also known as Dynamic Translation.



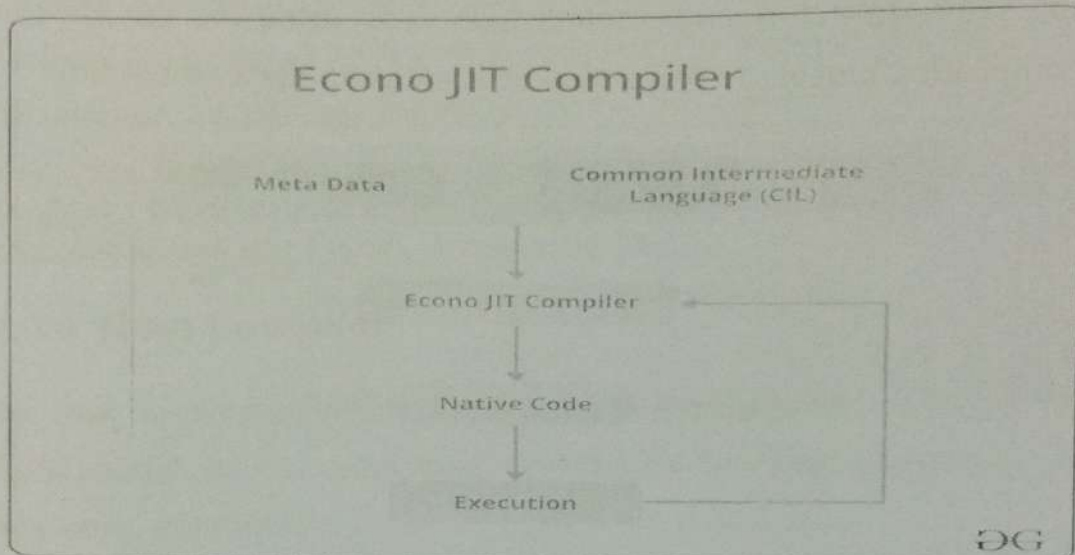
- **Types of Just-In-Time Compiler:** There are **3** types of JIT compilers which are as follows.
- **Pre-JIT Compiler** - All the source code is compiled into the machine code at the same time in a single compilation cycle using the Pre-JIT Compiler. This compilation process is performed at application deployment time.



- **Normal-JIT Compiler** - The source code methods that are required at run-time are compiled into machine code the first time they are called by the Normal JIT Compiler. After that, they are stored in the cache and used whenever they are called again.



- **Econo JIT Compiler** - The source code methods that are required at run-time are compiled into machine code by the Econo JIT Compiler. After these methods are not required anymore, they are removed.



- The JIT compiler requires less memory usage as only the methods that are required at run-time are compiled into machine code by the JIT Compiler.
- Page faults are reduced by using the JIT compiler as the methods required together are most probably in the same memory page.
- Code optimization based on statistical analysis can be performed by the JIT compiler while the code is running.

• JIT vs Traditional Compiler

Traditional Compiler

- **Compilation Process:** Compiles the entire source code into machine code before the program runs.
- **Execution:** Executes the pre-compiled machine code directly.
- **Advantages:** Faster start up time as the code is already compiled.
- **Disadvantages:** May not be optimized for specific runtime conditions or code paths. Can be less flexible for dynamic code changes.

JIT Compiler

- **Compilation Process:** Compiles bytecode into machine code during program execution. It identifies frequently used code sections (hotspots) and optimizes them.
- **Execution:** Executes the compiled machine code for hotspots, potentially interpreting other code sections.
- **Advantages:** Optimizes code based on runtime behavior, leading to better performance for long-running or frequently used code. Can be more flexible with dynamic code.

- **Disadvantages:** Initial startup time may be slower due to the compilation process. Requires more memory and CPU resources during runtime for compilation.

- **Namespace (Purpose and Type)**

- Namespace is a way to organize and group related code elements like classes, structs, enums, and interfaces, preventing naming conflicts and providing a hierarchical structure for your code.
- Purpose and type:
- **Organization:-** Namespaces logically group related code elements, making it easier to manage and understand large projects.
- **Avoiding Naming Conflicts:** They prevent naming collisions between classes, structs, etc., with the same name in different parts of your application or when using external libraries.
- **Hierarchical Structure:** Namespaces allow you to create a hierarchical structure for your code, improving readability and maintainability.

- ❖ **Type:-**

- **User-defined Namespaces:** These are namespaces created by the developer to organize their own code.
- **Built-in Namespaces:** The .NET Framework provides numerous built-in namespaces, such as System, System.IO, and System.Net, which contain classes for common functionalities.
- **Nested Namespaces:** Namespaces can be nested within other namespaces, creating a hierarchical structure for better organization, for example, System.IO.Compression.
- **File-scoped Namespaces:** C# 10 introduced the ability to declare a namespace for all types within a file without using curly braces, making code more concise.
- **Alias Namespaces:**

You can create aliases for namespaces (or types) using the using keyword, allowing you to refer to them with shorter, more convenient names, especially useful when working with long or ambiguous names.

❖ Common Type System. (CTS)

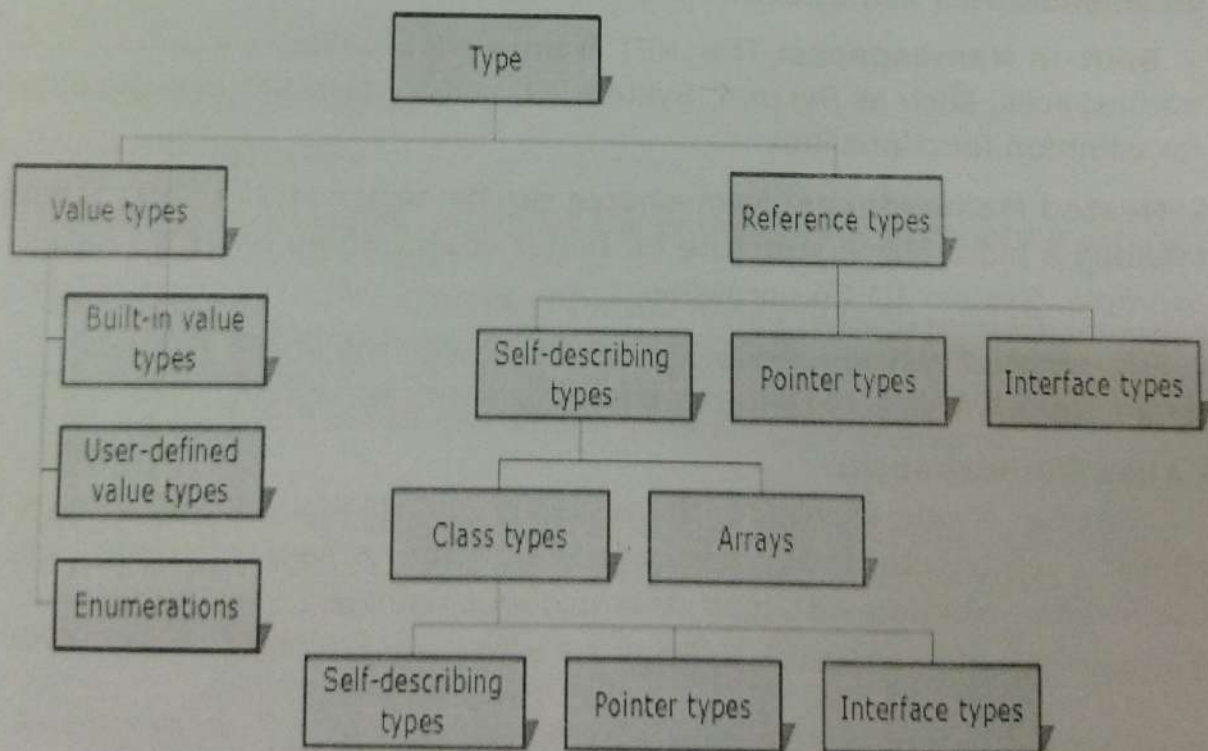
The common type system defines how types are declared, used, and managed in the runtime, and is also an important part of the runtime's support for cross-language integration.

The common type system performs the following functions:

- Establishes a framework that helps enable cross-language integration, type safety, and high performance code execution.
- Provides an object-oriented model that supports the complete implementation of many programming languages
- Defines rules that languages must follow, which helps ensure that objects written in different languages can interact with each other.

The common type system supports two general categories of types

- Value types
- Reference types



Common Type System(CTS).

Value types

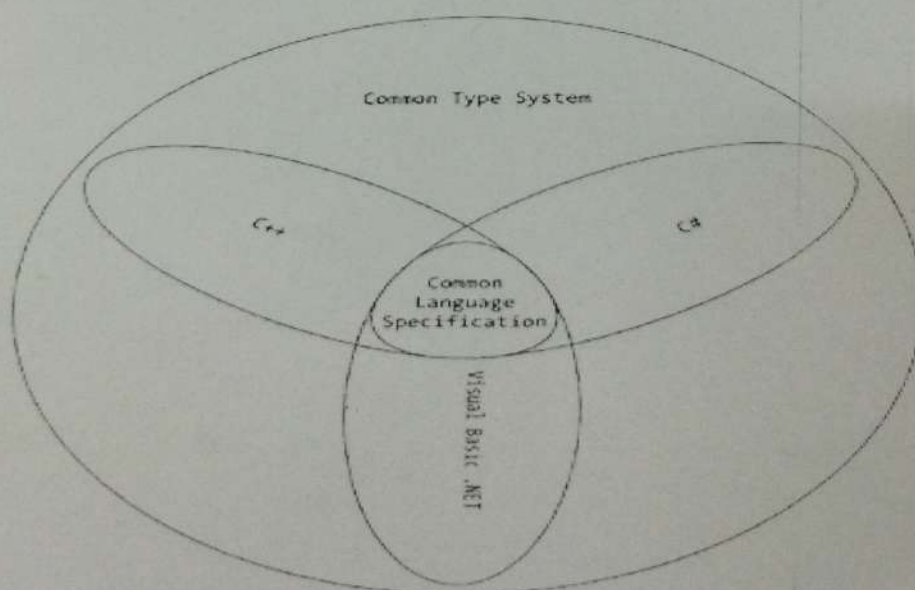
- Value types directly contain their data, and instances of value types are either allocated on the stack or allocated inline in a structure.
- Value types can be built-in (implemented by the runtime), user-defined, or enumerations.

Reference types

- Reference types store a reference to the value's memory address, and are allocated on the heap. Reference types can be self-describing types, pointer types, or interface types.
- the type of a reference type can be determined from values of self-describing types.
- Self-describing types are further split into arrays and class types. The class types are user-defined classes, boxed value types, and delegates.

❖ Common Language Specification. (CLS)

- The Common Language Specification (CLS), which is a set of basic language features needed by many .Net applications to fully interact with other objects regardless of the language in which they were implemented.
- This is a subset of the CTS which all .NET languages are expected to support.



CLS

- Most of the members defined by types in the .NET Framework Class Library (FCL) are Common Language Specification (CLS) compliant Types.

❖ **Class Loader:-**

- **Class Loader** is handled by the **Common Language Runtime (CLR)**. Unlike Java, where the "Class Loader" is a distinct concept, in .NET, the CLR takes care of loading types, assemblies, and metadata at runtime.
- Loads **assemblies** (DLLs or EXEs) into memory.
- Resolves **types** within those assemblies.
- Performs **Just-In-Time (JIT)** compilation.
- Ensures **type safety** and **security**.

❖ **Interoperability with Unmanaged Code**

- **Interoperability with Unmanaged Code** refers to the ability of managed code (such as C# or VB.NET running under the .NET CLR) to interact with unmanaged code, such as native C/C++ libraries or Windows API functions, that are not executed by the .NET runtime.
- **Managed code:-** Code executed by the .NET CLR (Common Language Runtime).
- **Unmanaged Code:-** Code executed directly by the OS, such as C/C++ code, Win32 API, COM components.
- **Interoperability:-** Mechanism that allows managed and unmanaged code to work together.
- **Platform Invocation Services:-** Used to call functions from unmanaged DLLs (e.g., Win32 API).
- **COM Interop:-** Enables interaction with **COM (Component Object Model)** components.
- Import COM library into .NET via add reference in Visual Studio.